M15 TDD / BDD / SDD Workflows

Méthodologie · Intermediate

Méthodologies de développement structurées avec Claude Code

Comparaison des trois approches

Aspect	TDD	BDD	SDD
Point de départ	Test technique	Comportement métier	Spécification écrite
Langage	Code de test	Given/When/Then	Document structuré
Audience	Développeurs	Devs + métier	Équipe entière
Feedback	Tests pass/fail	Scénarios validés	Spec implémentée

TDD avec Claude : Red-green-refactor

Le cycle classique, rendu explicite dans les prompts pour éviter que Claude ne saute directement à l'implémentation.

```
# Phase RED : demander le test qui échoue d'abord
> "Écris un test FAILING pour isValidEmail().
  La fonction n'existe pas encore."

# Phase GREEN : implémentation minimale
> "Écris le code minimal pour faire passer ce test."

# Phase REFACTOR : amélioration
> "Améliore l'implémentation. Les tests doivent
  rester verts."
```

Être explicite sur « FAILING » et « n'existe pas encore » compte, sinon Claude tend à écrire test et implémentation ensemble.

BDD avec Claude : Given/when/then

```
> "Décris le comportement de la page de login
  en scénarios Gherkin, puis implémente."

# Claude génère :
# Scénario: Connexion avec identifiants valides
#   Given un utilisateur enregistré avec email test@ex.com
#   When il soumet le formulaire avec le bon mot de passe
#   Then il est redirigé vers son tableau de bord
```

Les scénarios servent à la fois de spec, de documentation et de base de test, dans un langage lisible par des non-développeurs.

SDD avec Claude : Spec d'abord

Écrire la spécification complète avant la première ligne de code. Claude utilise ensuite la spec comme référence tout au long de l'implémentation.

```
> "Voici la spec de l'endpoint POST /auth/login
  [coller la spec]. Implémente section par section
  en respectant chaque contrainte."
```

Verification Loops : le principe fondateur

Les trois méthodes reposent sur le même principe identifié par Boris Cherny : **un agent qui peut «voir» ce qu'il a produit donne de meilleurs résultats.** Sans mécanisme de vérification, Claude itère à l'aveugle.

Domaine	Outil de vérification
Backend	Tests (unit, integration)
Frontend	Aperçu navigateur live
Types	Compilateur TypeScript
Style	ESLint / Prettier
Sécurité	Semgrep, analyseurs statiques

Chain-of-verification

Pattern de validation croisée (arXiv:2309.11495) pour réduire les hallucinations : Claude génère une réponse, puis génère des questions de vérification sur cette réponse, puis répond à ces questions pour détecter les incohérences.

Applicable quand la correction du résultat est critique et que les tests automatiques ne suffisent pas (algorithmes complexes, logique métier subtile).

Intégration avec Tasks API

Activer les outils de tâches avec `CLAUDE_CODE_ENABLE_TODO_TOOLS=1`, puis demander à Claude :

```
Create tasks for failing auth tests, implementation, and refactoring.
Use the IDs returned by TaskCreate to add dependencies with
TaskUpdate:
implementation waits for tests; refactoring waits for implementation.
```

Le cycle Red-Green-Refactor devient traçable et persistant entre sessions.